North South University

Department of Electrical and Computer Engineering

Software Quality Assurance Report

Design and Implementation of Playwright JavaScript Automation Framework for E-Commerce Web Testing

---

Shah Nadim Kamran Rian

Department of Electrical and Computer Engineering
North South University

shah.rian.252@northsouth.edu

Abstract

This project describes how Software Quality Assurance (SQA) practices were applied to the Automation Exercise e-commerce platform. The project started with exploratory manual testing to understand the application behavior, then moved on to designing and documenting structured manual test cases. After identifying the most important risks, selected critical test scenarios were chosen for automation.

The automation framework was developed using Playwright with JavaScript, with emphasis on modular design, maintainability, and reuse. The framework supports user interface (UI), application programming interface (API), and end-to-end (E2E) testing. It also includes structured test execution, tagging strategy, and reporting support.

The framework executed over 98 automated test cases with stable results. The results show that automation improves testing efficiency by reducing execution time and improving repeatability and accuracy. The project also identifies the main challenges faced during implementation and suggests future improvements.

Overall, this project demonstrates a complete SQA lifecycle by combining manual and automated testing to support reliable quality validation of a web-based application.

Contents

Abstract **1**

1 Introduction **3**

2 Project Goals **3**

3 Application Under Test **3**

4 Manual Testing Methodology **4**

4.1 Manual Test Design Techniques 4

5 Manual-to-Automation Framework Workflow **4**

5.1 Steps Followed to Build the Automation Framework 5

6 Automation Framework Design **5**

7 Automation Coverage and Growth **6**

8 Test Tagging Strategy **8**

9 Automation Selection Rationale **8**

10 Execution Results **9**

11 Key Findings **12**

11.1 Functional Findings 12

11.2 API Findings 12

11.3 Automation Framework Findings 13

11.4 Manual Testing Findings 13

12 Difficulties and Mitigations **13**

12.1 Public Website Timing and Stability 13

12.2 Advertisements and Third-Party Scripts 13

12.3 Selector Reliability 14

12.4 Unique Test Data 14

12.5 Flaky Checkout, Payment, and Invoice Tests 14

12.6 Manual-to-Automation Gap 14

13 Limitations **15**

14 Future Work **15**

15 Conclusion **15**

References **16**

1 Introduction

Software testing is vital in software development. It ensures an application works under both expected and unexpected conditions. For this project, the Automation Exercise website was chosen for testing. The site offers many realistic e-commerce features, including user registration, login, product search, product detail pages, cart functionality, checkout, contact form, file upload, subscription, invoice download, and API endpoints.

The project began with manual testing, followed by automation efforts. Manual testing was prioritized to build an understanding of website functionality, determine key user flows, identify expected verification results, and select relevant test cases for regression testing. Based on this analysis, I developed a Playwright JavaScript framework to automate stable, repeatable, high-value test cases.

2 Project Goals

- Perform deep manual testing before building automation scripts.
- Write structured manual test cases in an Excel workbook with feature-wise sheets.
- Identify smoke, regression, negative, boundary, API, and end-to-end scenarios.
- Build a Playwright JavaScript automation framework from scratch.
- Automate selected high-priority and frequently used manual test cases.
- Maintain traceability between manual test cases and automated spec files.
- Use test tags such as @smoke, @regression, @api, @negative, @e2e, and @slow.
- Debug and stabilize tests that failed or became flaky due to public-site timing issues.
- Prepare the repository for GitHub.

3 Application Under Test

I tested Automation Exercise, a public e-commerce practice website. This site was a good fit for my project because it offers both user interface features and API endpoints. It let me try out various SQA activities, including functional UI testing, form validation, negative testing, regression testing, API testing, and end-to-end user flows.

Table 1. Manual Testing Scope

Module	Manual Testing Scope
Homepage	Page load, title, header links, footer, scroll behavior, subscription area.
Signup/Login	New registration, invalid login, duplicate signup, logout, password masking, field validation.
Products	Product list, search, product details, price, category, brand, availability, review form.
Cart	Add product, multi-item cart, remove item, quantity, price, total, checkout button.
Checkout	Guest checkout prompt, logged-in checkout, address details, order review, payment, invoice.
Contact	Contact form, file upload, invalid email, browser alert, success message.
API	Products API, brands API, search API, invalid methods, missing parameters.
Combined Flows	Signup to logged-in state, search to cart, guest checkout, registered checkout.

4 Manual Testing Methodology

I started by manually testing the website to see how it worked for users. While exploring the app, I wrote down test cases in an Excel file. This file had details like test case IDs, parts of the app, priorities, test steps, expected and actual results, pass or fail status, smoke or regression labels, and whether each test could be automated. This manual test plan later became the base for the automation system.

4.1 Manual Test Design Techniques

- For positive testing, I tried actions such as signing up with correct information, searching for products, adding items to the cart, submitting the contact form, and making successful payments.
- Negative testing involved things like entering incorrect login details, using invalid email formats, trying to sign up multiple times with the same information, calling unsupported API methods, and missing required API parameters.
- For boundary and validation testing, I checked things like leaving fields empty, making sure required fields were filled, testing email input types, verifying password masking, and searching with special characters.
- End-to-end testing involved running through complete flows, such as searching for a product, adding it to the cart, checking out, making a payment, and generating an invoice.
- For regression classification, I marked test cases that need to be run again after future updates as regression candidates.
- For smoke classification, I identified the most important tests that quickly check if the main parts of the application are working and marked them as smoke candidates.

5 Manual-to-Automation Framework Workflow

Manual testing was done before automation to learn how the application works, how users move through it, and what results to expect. After creating the manual test cases, the best ones for automation were chosen based on how stable, repeatable, often run, and important for regression they were. Not all manual test cases were automated. The manual tests were the main guide, and only the most important scenarios with clear steps and checkable expected results were automated using Playwright.

Project visuals were created from the Playwright files and the step-by-step development plan. They show how automation grew, coverage by module, how manual tests were converted to automated ones, and main challenges faced.

The overall workflow followed in this project is shown in Fig. 1.

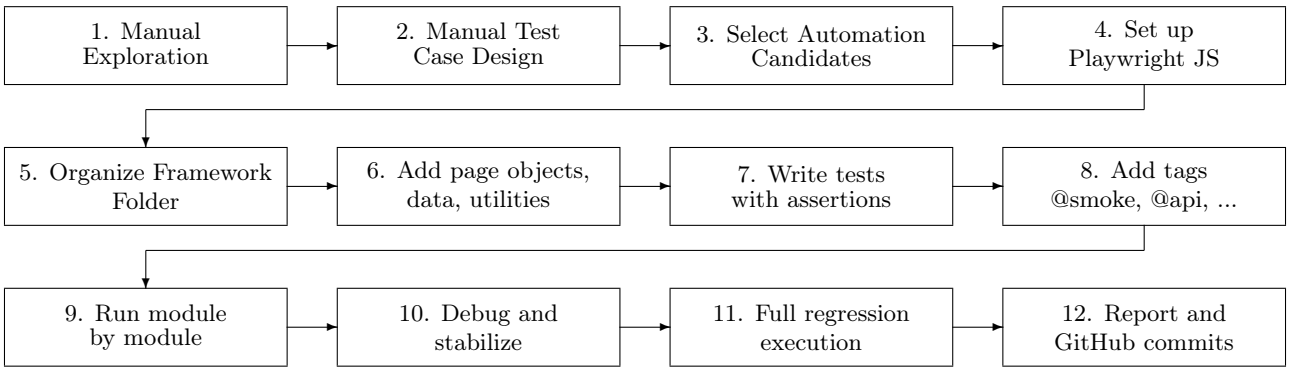

Figure 1: Workflow followed to build the Playwright automation framework from manual testing.

This figure shows the step-by-step process followed in the project, starting from manual exploration and test case design, then moving into automation candidate selection, framework setup, test implementation, debugging, reporting, and GitHub documentation

5.1 Steps Followed to Build the Automation Framework

1. Manually explored the website modules and real user journeys.
2. Prepared manual test cases in Excel with module, test steps, expected results, priority, and status.
3. Marked cases as smoke, regression, negative, boundary, API, or end-to-end.
4. Selected automation candidates based on frequency, stability, repeatability, business importance, and clear expected results.
5. Initialized the Playwright JavaScript project using Node.js and npm.
6. Created a professional folder structure with tests, pages, test-data, utilities, configuration, and README files.
7. Implemented UI, API, negative, cart, checkout, contact, product, and end-to-end spec files.
8. Added assertions to verify URLs, text, visibility, API response content, cart rows, form fields, and payment confirmation.
9. Added tag planning for @smoke, @regression, @api, @negative, @e2e, and @slow execution groups.
10. Ran each test file individually, fixed failures, and then executed the full regression suite.
11. Used Playwright HTML reports, trace evidence, and screenshots for debugging.
12. Committed the project gradually to GitHub to show continuous development.

6 Automation Framework Design

The automation system was created using Playwright with JavaScript. It uses a modular design so each part of the website has its own test files. This makes the system easier to understand, fix, update, and add to. The design also shows the project was made as a real test automation system instead of just a set of random scripts.

Table 2. Automation Framework Structure

Folder/File	Purpose
manual-testing/	Contains manual test case workbooks, traceability, bug report, and coverage documents.
automation-testing/tests/	Contains Playwright spec files grouped by functional module.
automation-testing/pages/	Contains reusable page-object style actions and locators.
automation-testing/test-data/	Stores reusable test data and upload files.
automation-testing/utils/	Stores helper functions such as random email generation.
playwright.config.js	Defines test settings, browser execution, reporters, screenshots, traces, and timeouts.
package.json	Stores dependencies and project scripts.
README.md	Explains project goal, setup, execution commands, and coverage.

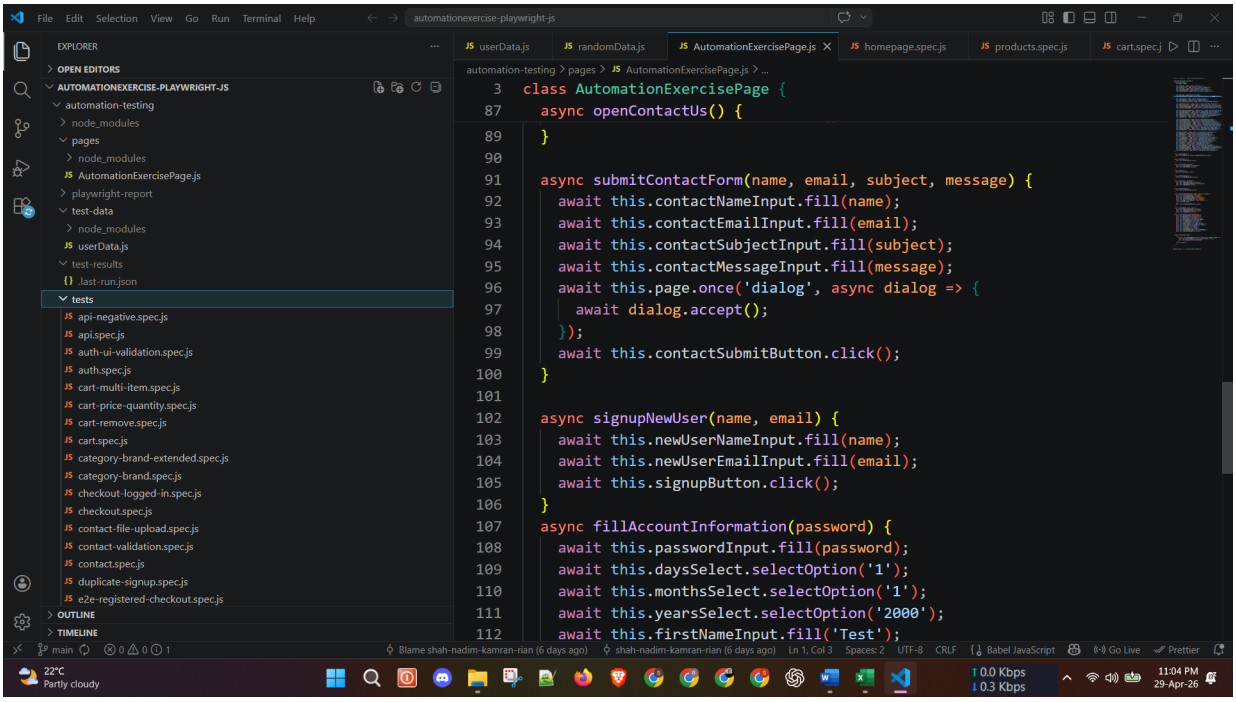

Figure 2: Automation framework showing tests, pages, test-data, utils, package.json, and playwright.config.js.

This figure shows the actual project structure in Visual Studio Code. It highlights how test files, page objects, test data, and utility functions are organized into separate folders, supporting a modular and maintainable automation framework.

7 Automation Coverage and Growth

The automation suite was built step by step. In Phase 1, we set up the basic framework. Phase 2 added login features and static pages. In Phase 3, we worked on product search, product details, categories, brands, and reviews. Phase 4 brought in advanced features like checkout, payment, invoicing, and full user workflows. By working in stages, we could find and fix issues in each part before moving on.

Module	Test Cases	Smoke	Regression	Playwright Ready
Homepage	18	6	12	18
Signup/Login	40	6	34	40
Products	24	4	20	24
Cart	16	3	13	16
Checkout	12	3	9	12
Contact Us	12	1	11	12
Combined Flows	15	3	12	15

Figure 3: Automation Coverage Distribution Across Application Modules.

This figure shows the number of total tests, simple checks, repeated checks, and test scenarios that can be automated in each part of the system. The Signup/Login and Products parts have the most tests, showing they matter a lot to how the system works.

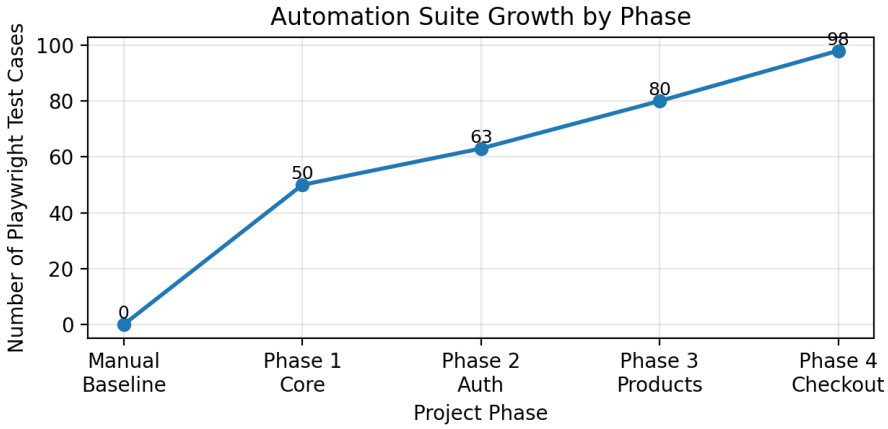

Figure 4: Automation suite growth by project phase.

This figure shows the steady growth in automated test cases through the project phases. The test suite grew from a starting point to 98 tests, showing the step-by-step addition of important parts like login, product checks, and checkout processes.

Table 3. Automation Phase Summary

Phase	Target Tests	Main Coverage
Phase 1	50	Core baseline: homepage, products, search, cart, contact, API, subscription, footer.
Phase 2	63	Auth UI validation, logout/session, duplicate signup, static pages.
Phase 3	80	Advanced search, product extra checks, extended category/brand, review validation.
Phase 4	98	Logged-in checkout, payment, invoice download, registered end-to-end checkout.

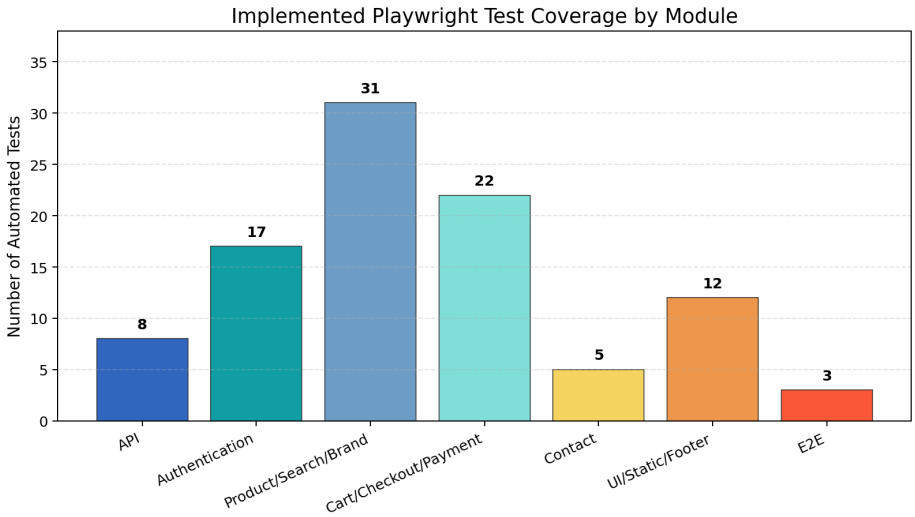

Figure 5: Implemented Playwright Test Coverage by Module.

This figure shows how Playwright test cases are spread across different parts of the system. More tests are done in Product/Search/Brand and Cart/Checkout areas because they are very important for the

main e-commerce functions. Fewer tests in API and E2E parts show that automation is focused and done based on testing needs.

8 Test Tagging Strategy

Test tags such as @smoke, @regression, @api, @negative, @e2e, and @slow were defined to enable flexible and selective test execution.

Table 4. Test Tagging Strategy

Tag	Purpose	Example Use
@smoke	Quickly verifies core application health.	Homepage, navigation, product search, cart entry, API positive checks.
@regression	Executes stable, broad test coverage after changes.	Auth, products, cart, checkout, contact, subscription, API.
@api	Runs service-level tests separately from UI tests.	Products list, brands list, search API, API negative checks.
@negative	Validates error handling and validation behavior.	Invalid login, duplicate signup, invalid email, missing API parameters.
@e2e	Marks long user journey tests.	Guest checkout, registered checkout, payment, invoice flow.
@slow	Identifies tests that require extended execution time.	Logged-in checkout, payment, invoice, full registered checkout.

This tagging strategy improves test organization and allows efficient execution of targeted test suites during development and regression cycles.

9 Automation Selection Rationale

Not all manual cases were automated. Automation works best for stable, repeatable, high-priority tests. Manual testing covers subjective, unstable, or low-impact cases.

- Visual layout checks and subjective usability observations are better evaluated manually.
- Some exploratory cases require human judgment and cannot be verified reliably with simple assertions.
- Public-site advertisements, network speed, and dynamic behavior can introduce instability in automated execution.
- Aggressive security or performance testing should not be performed against a public practice website.
- Very destructive or repetitive account/order scenarios can create unnecessary test data accumulation.
- Low-priority UI checks may not provide enough return on automation effort.

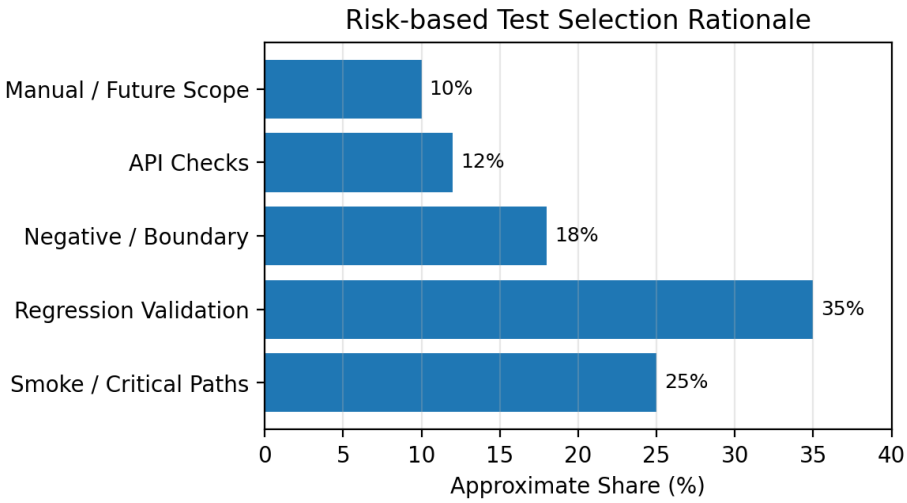

Figure 6: Conceptual risk-based automation selection rationale.

This figure presents a conceptual distribution of test case categories considered during automation selection. Regression and critical path scenarios form the largest portion, while API, negative, and manual-only cases represent more targeted or specialized testing areas.

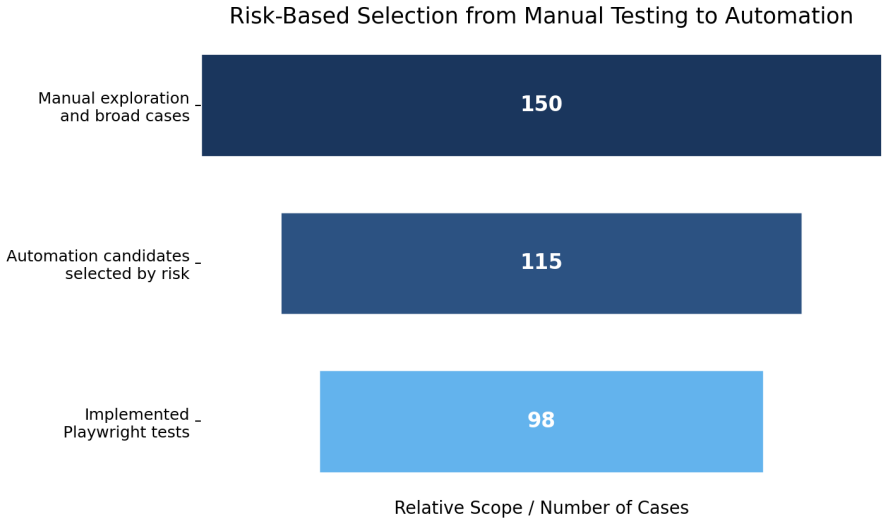

Figure 7: Risk-Based Selection from Manual Testing to Automation.

This figure illustrates how broad manual test coverage was progressively reduced to a focused set of automation candidates and finally implemented Playwright test cases based on risk and priority.

10 Execution Results

The automated tests were run locally using Chromium with Playwright. Tests were run one module at a time first, then the whole set was run together. The project grew to 98 tests spread over 36 files. During development, some tests failed or became unreliable, especially in checkout and payment steps. These problems were fixed by using better ways to find elements, waiting clearly for actions, using unique data, adjusting time limits, and running long processes one after another.

Table 5. Execution Results

Execution Metric	Result
Manual test cases documented	137
Smoke cases	26
Regression cases	111
Automation candidates documented	137
Automated Playwright tests implemented	98
Spec files used in automation suite	36
Final full-suite result	98 passed
Approximate final execution time	3.4 minutes
Primary browser used	Chromium
Reporting mechanism	Playwright HTML report, terminal output, screenshots, and trace evidence

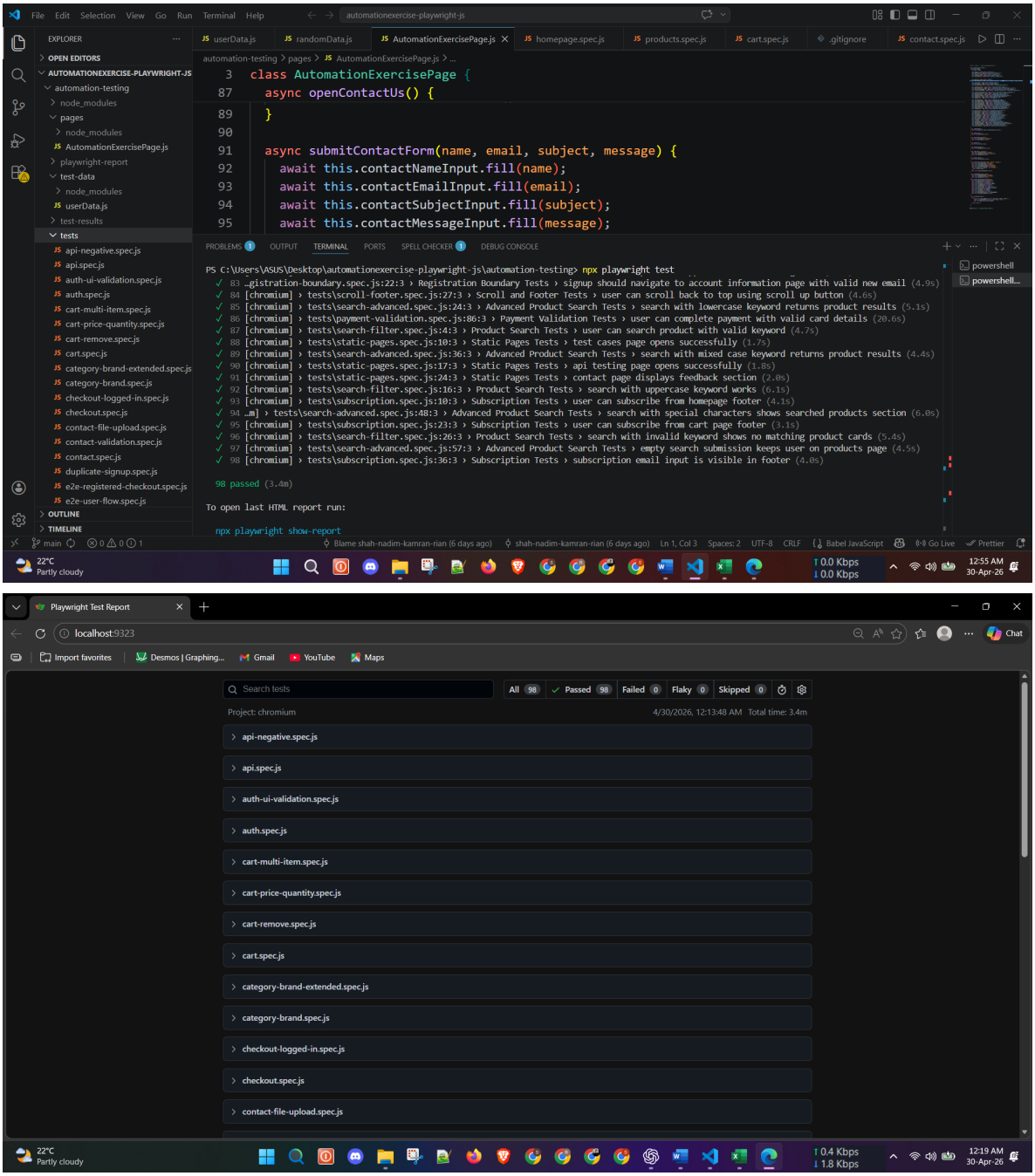

Figure 8: Latest Playwright HTML report screenshot showing the highest stable passed result.

This figure provides execution evidence from the Playwright HTML report. It shows the final stable run of the automation suite and supports the reported result of 98 passed tests.

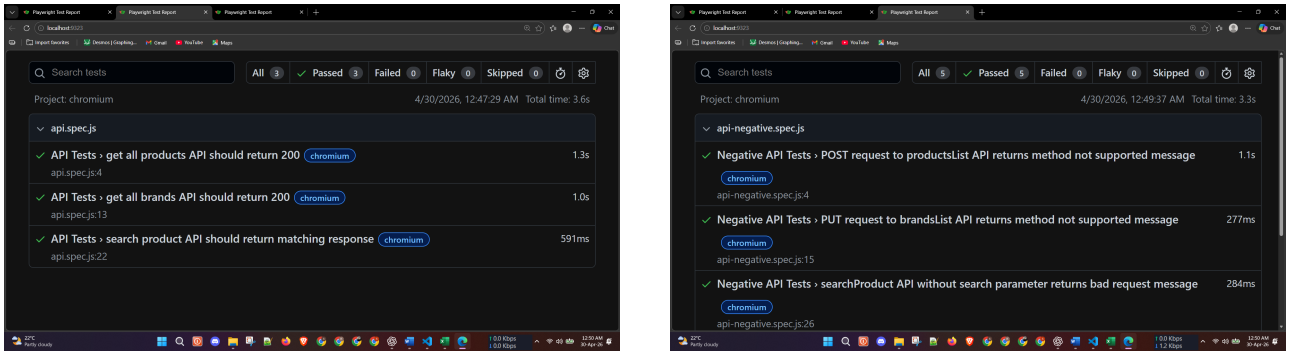

Figure 9: Screenshot of API and API-negative tests in the Playwright report.

This figure shows evidence of API-level testing, including positive and negative API scenarios. It supports the claim that the framework covered both UI testing and service-level validation.

11 Key Findings

The findings from this project are important because they show what was learned about the website behavior and the automation framework, not only whether tests passed. The main findings are discussed below.

11.1 Functional Findings

The website contains many realistic e-commerce user flows. Homepage navigation, product listing, product search, product details, add-to-cart, remove-from-cart, checkout prompt, contact form submission, and subscription features were testable with Playwright. Basic navigation and static pages were generally stable when tested using URL assertions and body text assertions. Product modules were also strong automation candidates because expected results such as product cards, prices, categories, brands, and search results could be verified clearly.

Authentication features required more careful handling. New user registration worked, but automated tests required unique email addresses to avoid duplicate-account failures. Invalid login, invalid email, duplicate signup, logout, and password masking were useful negative and validation checks. These tests showed that the site provided visible error messages for common invalid actions and that login/signup input fields could be verified through DOM attributes and visible text.

Cart and checkout behavior showed both stable and timing-sensitive results. Simple guest cart and guest checkout prompt tests were reliable after using stronger locators and explicit waits. Multi-item cart tests and price/quantity checks were useful because they verified important e-commerce data in the cart table. Logged-in checkout and payment flows were functionally possible, but they were more sensitive to timing because they involve many sequential actions.

11.2 API Findings

API testing provided faster validation compared with UI tests. Positive API tests for product list, brand list, and product search were useful for verifying service-level availability. Negative API tests were also valuable because unsupported HTTP methods and missing parameters returned predictable response messages. These tests were good candidates for the @api and @negative tags because they are faster and less dependent on browser rendering.

11.3 Automation Framework Findings

The framework became more maintainable when tests were grouped by module. Separate spec files for login, product search, category/brand, cart, checkout, contact, API, and end-to-end flows made the project easier to debug. The project also showed that automation should be added gradually. Running one module at a time made it easier to identify which file caused a failure. After individual files passed, the full suite could be executed with more confidence.

The use of stable locators was one of the most important findings. Selectors based only on broad visible text sometimes matched duplicate elements. More reliable selectors included data-qa attributes, href-based navigation links, scoped product cards, and cart table rows. Blocking advertisement-related requests also improved stability because ads sometimes changed page loading or interfered with clicking elements.

11.4 Manual Testing Findings

Manual testing proved necessary because it revealed which flows were important and how users actually move through the site. The manual workbook helped me identify smoke candidates, regression candidates, negative cases, and future-scope areas. It also helped explain why the automated suite does not need to be identical to the manual suite. Manual testing gave broader coverage, while automation focused on repeatable checks that provide the highest value during regression.

- Navigation, product search, cart, contact, and API checks were the most stable automation candidates.
- Signup and checkout tests were valuable but needed unique test data and longer waits.
- Payment and invoice tests were possible but should be marked as @slow due to long execution time.
- API tests were faster and more stable than browser-based checkout tests.
- Manual testing remained important for visual layout, usability, exploratory observation, and low-priority UI checks.

12 Difficulties and Mitigations

The biggest challenge was not just writing Playwright code. Making the tests reliable enough to run over and over was much harder. Since the application is a public practice website, I could not control things like server speed, ads, outside scripts, or cleaning up test data. Here are the main problems I faced during the project.

12.1 Public Website Timing and Stability

Certain pages occasionally loaded more slowly, which resulted in tests failing when Playwright attempted to use an element that wasn't ready. Because checkout, payment, and invoice processes involve numerous steps—such as creating an account, adding products, browsing the basket, completing payment forms, and confirming the order—this issue was particularly evident in these testing. I fixed issue by making sure the correct page information appeared before proceeding, adding obvious checks before actions, and giving additional time for complicated procedures.

12.2 Advertisements and Third-Party Scripts

The website occasionally loads relevant scripts or advertisements. These could alter how clicks function, move things around, or slow down the page. As a result, even when the test procedures were correct, some tests became unreliable. I resolved this by using page.route to ban popular ad websites in many test files. As a result, there was less unnecessary network activity and the tests were more repeatable.

12.3 Selector Reliability

I started using getByText and other broad text selections. When the same text appeared in multiple locations, they didn't always work. For instance, the label "Add to cart" may be used on both product cards and overlay buttons. I used more focused locators, data-qa tags, href selectors for navigation, and cart table row selectors to increase the tests' dependability. As a result, there were fewer false matches and the tests were more accurate.

12.4 Unique Test Data

Sign-up and checkout testing need unique email addresses. The website would display a duplicate email error if I repurposed an email. I used Date.now and occasionally random numbers to build email addresses in order to fix issue. This let me run account creation tests repeatedly without having to clean up old users manually.

12.5 Flaky Checkout, Payment, and Invoice Tests

Checkout, payment, and invoice tests were the most difficult because they involved lengthy start-to-finish processes. Even if the program functioned properly, subsequent processes could fail if one step was slow. I ran lengthy flows sequentially, allowed extra time for steps, waited for the appropriate payment and invoice components, and used the planned @slow tag to distinguish between slow and quick tests in order to make these tests less unstable. This covered more checkout details while maintaining a quick main test set.

12.6 Manual-to-Automation Gap

Although the manual test workbook contained a large number of test cases, not all of them could be automated. Some manual testing examined low-priority details, exploratory actions, user experience, and visual alignment. The test set would grow but not improve if things were automated. I dealt with this by putting down guidelines for deciding what to automate and outlining which cases were saved for later or remained manual.

Table 6. Difficulties and Mitigation Strategies

Difficulty	Observed Problem	Mitigation
Public-site timing	Pages loaded differently across runs, especially checkout and payment.	Used explicit waits, stronger assertions, and longer timeouts for heavy flows.
Advertisements	Third-party ads slowed pages and sometimes affected clicks.	Blocked common advertisement domains using page.route.
Duplicate selectors	Broad text selectors matched repeated elements.	Used data-qa, href, scoped product cards, and table row locators.
Account data	Duplicate emails caused signup failures.	Generated unique emails using timestamps and random values.
Long E2E flows	Payment and invoice tests became flaky under slow loading.	Used serial mode, @slow planning, and exact element waits.
Manual/automation mismatch	Some manual cases needed human judgment.	Used risk-based automation selection and documented manual-only scope.

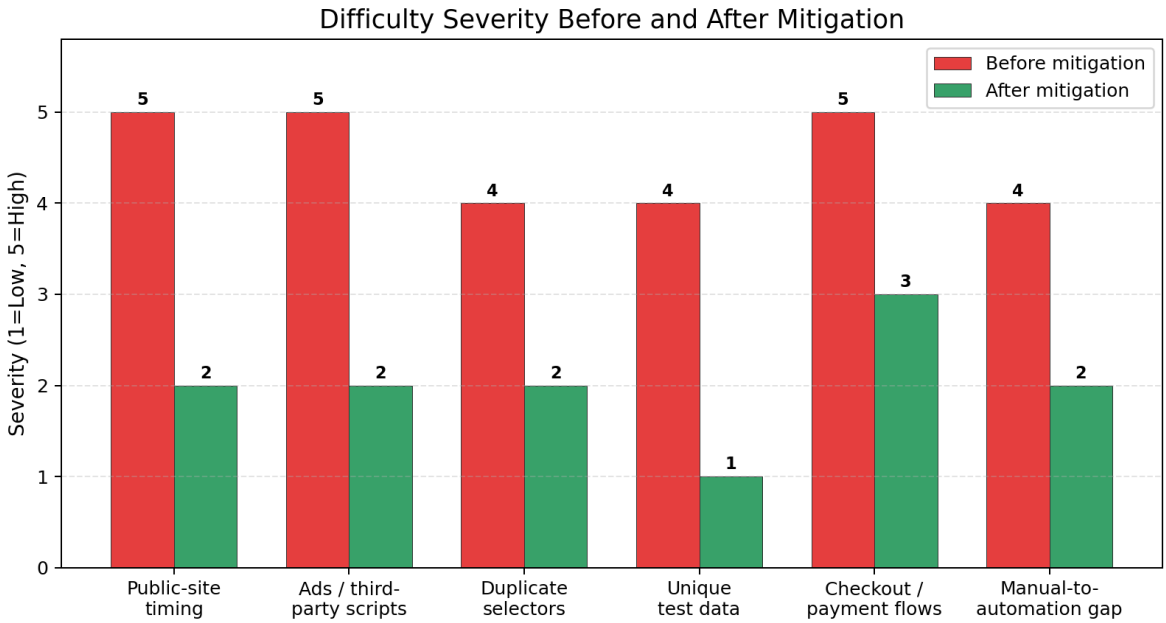

Figure 10: Difficulty Severity Before and After Mitigation.

This figure shows the reduction in severity of major testing challenges after applying mitigation strategies. Significant improvements were observed in timing issues, selector reliability, and test data handling, indicating increased stability of the automation framework.

13 Limitations

i. The website is public, so the test environment is not controlled by me.
ii. Some checkout and payment flows may remain timing-sensitive depending on network and server load.
iii. Full load testing and aggressive security testing were not performed because the application is public.
iv. Visual regression testing was not fully implemented in this version.
v. CI execution through GitHub Actions can be added in future work.
vi. Some manual cases remain manual because they require human visual judgment or exploratory observation.

14 Future Work

- Add GitHub Actions workflow for automatic smoke and regression execution.
- Refactor repeated signup, checkout, and payment actions into reusable helper classes.
- Add cross-browser tests using Chromium, Firefox, and WebKit.
- Add responsive viewport tests for mobile and tablet layouts.
- Add API schema validation for products, brands, and search responses.
- Add visual comparison tests for stable pages if the environment becomes controlled.
- Update the automation traceability workbook after each new automation phase.

15 Conclusion

This project demonstrates a complete SQA workflow from manual testing to automated regression coverage. The Automation Exercise website was first explored manually and documented feature-wise

test cases. The most suitable automation candidates were then selected and implemented within a Playwright JavaScript framework. The automation suite covered homepage, authentication, product search, product details, cart, checkout, contact form, subscription, API, and end-to-end flows.

The project also showed that good automation is not only about increasing test count. Stable automation requires understanding the product, using reliable selectors, generating unique test data, handling timing issues, and deciding which tests should remain manual. The findings and difficulties from this project helped improve the framework and made the final report more realistic. Overall, the work shows a structured and professional approach to software quality assurance.

References

1. Automation Exercise, "Full-Fledged Practice Website for Automation Engineers." Available: https://automationexercise.com/. Accessed: Apr. 19, 2026.
2. Automation Exercise, "Automation Practice Website for UI Testing – Test Cases." Available: https://automationexercise.com/test_cases. Accessed: Apr. 21, 2026.
3. Microsoft Playwright, "Playwright Test Documentation." Available: https://playwright.dev/docs/intro. Accessed: Apr. 26, 2026.
4. IEEE, "IEEE Editorial Style Manual for Authors." Available: https://journals.ieeeauthorcenter.ieee.org/. Accessed: Apr. 30, 2026.